AGENDA 1

UTILIZANDO MICROSOFT MAUI PARA DESENVOLVIMENTO MOBILE

GEEaD - Grupo de Estudos de Educação a Distância

Centro de Educação Tecnológica Paula Souza

GOVERNO DO ESTADO DE SÃO PAULO

EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO

CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

PROGRAMAÇÃO MOBILE I

Expediente

Autores:

KELLY CRISTIANE DE OLIVEIRA DAL POZZO

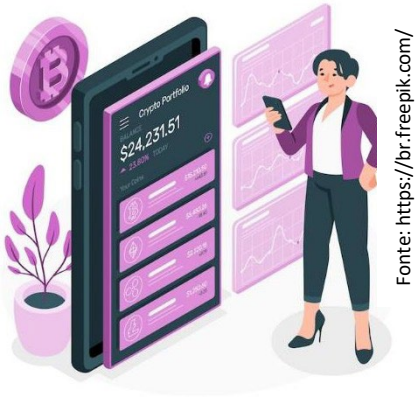
TIAGO ANTONIO DA SILVA

Revisão Técnica:

GUILHERME HENRIQUE GIROLLI

São Paulo – SP, 2024

Como já vimos no módulo anterior, o desenvolvimento mobile é uma área em crescente expansão no mercado de TI, permitindo que aplicativos atendam às necessidades mais variadas. No entanto, um aspecto fundamental para o sucesso dos aplicativos é a capacidade de armazenar e gerenciar os dados de forma eficiente e segura. Durante esta agenda, vamos iniciar o conceito de persistência de dados e como implementá-lo utilizando o SQLite no .NET MAUI, utilizando pacotes NuGet. Além disso, entenderemos a estrutura CRUD, essencial para operações básicas em bases de dados.



Fonte: https://br.freepik.com/

Persistência de Dados

A capacidade de salvar informações de forma que elas permaneçam disponíveis mesmo após o encerramento de uma aplicação é conhecida como persistência de dados. Este conceito é fundamental para qualquer sistema que precise manter dados a longo prazo, como listas de compras, registros financeiros ou histórico de interações do usuário.

Imagine a utilização de um aplicativo de controle de despesas, onde se gerencia os gastos diários de uma residência por exemplo, ao utilizar o aplicativo o usuário espera que suas transações permaneçam salvas e acessíveis mesmo após reiniciar o dispositivo. Sem persistência, as informações geradas durante o uso de um aplicativo seriam perdidas ao fechar a aplicação.

Persistência de dados significa que as informações geradas ou manipuladas durante o uso de um sistema ou aplicativo são armazenadas de maneira permanente ou pelo menos até que sejam removidas deliberadamente. Essa prática assegura que dados importantes não sejam perdidos quando o aplicativo é fechado ou o dispositivo reiniciado. Por exemplo, em um aplicativo de mensagens, as conversas entre os usuários devem estar acessíveis mesmo após o fechamento do aplicativo. Da mesma forma, sistemas de gestão, como controle de estoque ou finanças, precisam preservar os dados para garantir a continuidade e a confiabilidade das informações.

Como funciona a persistência de dados?

A persistência de dados pode ser implementada de várias maneiras, dependendo das necessidades do sistema:

- ✓ **Arquivos locais:** Simples e adequado para pequenos volumes de dados, como preferências do usuário.
- ✓ **Bancos de dados locais:** Como o SQLite, utilizado em dispositivos móveis para armazenar dados estruturados.
- ✓ **Serviços em nuvem:** Para sincronizar e compartilhar dados entre dispositivos.

No .NET MAUI, a persistência de dados é implementada com o SQLite, sendo um banco de dados relacional leve que permite salvar informações de forma estruturada e acessá-las rapidamente.

Base da Manipulação de Dados: CRUD

A persistência de dados é o ponto de partida para compreender como aplicativos gerenciam informações de forma duradoura. Depois de entender como os dados podem ser armazenados de forma confiável, é natural explorar as operações que possibilitam sua manipulação. Esse conjunto de operações, conhecido como CRUD. CRUD é um conceito essencial para o desenvolvimento de qualquer aplicação que interaja com dados armazenados. Ele define as quatro operações básicas que permitem criar, visualizar, atualizar e excluir informações de forma estruturada em um sistema ou banco de dados.

O termo CRUD é um acrônimo para:

- ✓ **Create (Criar):** Operação que insere novos dados em um sistema ou banco de dados.
- ✓ **Read (Ler):** Permite a visualização de informações armazenadas.
- ✓ **Update (Atualizar):** Atualiza registros existentes para refletir mudanças ou correções.
- ✓ **Delete (Excluir):** Remove registros que não são mais necessários.

Essas operações são fundamentais em praticamente todas as aplicações que lidam com armazenamento de dados, como sistemas bancários, lojas virtuais e redes sociais.

Para entendermos melhor o conceito, imagine um sistema de gestão de estoque, onde:

Create: Um administrador adiciona um novo produto ao inventário com informações como nome, preço e quantidade.

Read: Os usuários visualizam uma lista dos produtos disponíveis na loja.

Update: O administrador altera o preço ou a quantidade de um produto para refletir novas condições de mercado.

Delete: Produtos descontinuados são removidos do sistema.

O objetivo desta agenda é iniciar o desenvolvimento de um aplicativo para gerenciamento de compras, denominado AppMinhasCompras, durante o desenvolvimento do aplicativo para aplicar a persistência de dados vamos conhecer o SQLite, que irá realizar o armazenamento de dados localmente no App.

Mas oque é SQLite?

SQLite Banco de Dados local

O SQLite é um sistema de gerenciamento de banco de dados relacional (RDBMS) que se destaca por ser leve, embutido e autossuficiente, ou seja, não precisa de um servidor separado para funcionar. Ele é amplamente utilizado em dispositivos móveis, aplicativos desktop, IoT, e até mesmo em navegadores da web devido à sua simplicidade e eficiência.

Vejamos as características do SQLite que o tornam interessante para o desenvolvimento do nosso aplicativo:

Autossuficiência: O SQLite é um banco de dados baseado em arquivos. Ele armazena todas as informações, incluindo tabelas, índices e esquemas, em um único arquivo.

Sem Servidor: Diferentemente de outros RDBMS como MySQL ou PostgreSQL, o SQLite não requer um processo de servidor separado. A aplicação acessa diretamente o arquivo do banco de dados.

Leve e Compacto: O binário completo do SQLite é extremamente pequeno, geralmente menor que 1 MB, tornando-o ideal para dispositivos com recursos limitados.

Portabilidade: Os arquivos do SQLite podem ser copiados entre diferentes plataformas sem a necessidade de conversões ou adaptações.

Padrão SQL: Ele suporta grande parte do SQL padrão, incluindo comandos como SELECT, INSERT, UPDATE, e DELETE.

Zero Configuração: Não requer instalação, configuração ou administração. Basta incluir o arquivo do banco de dados no projeto e começar a usá-lo.

Confiabilidade: O SQLite utiliza transações ACID (Atomicidade, Consistência, Isolamento e Durabilidade), garantindo integridade de dados, mesmo em casos de falhas.

Desempenho: Por ser leve e trabalhar diretamente com arquivos, o SQLite é muito rápido para operações que envolvem dados de pequeno a médio porte.

Saiba mais sobre o SQlite acessando sua documentação:
https://sqlite.org/about.html

Apesar das vantagens e ótimas funcionalidades para atender a maioria dos Apps, o SQLite apresenta algumas limitações, como:

Multiusuário Limitado: O SQLite não é adequado para sistemas que precisam de alta concorrência, como aplicativos web de grande escala.

Limitações de Tamanho: Apesar de suportar bases de dados de até 140 TB, o SQLite não é otimizado para bancos muito grandes.

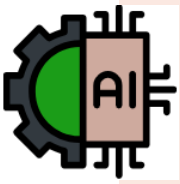
Recursos Avançados: Falta suporte para algumas funcionalidades presentes em bancos como PostgreSQL, como procedimentos armazenados ou replicação.

Configuração do SQlite no .NETMAUI

O .NET permite o uso de pacotes que facilitam a rotina de desenvolvimento. Mas o que são pacotes?

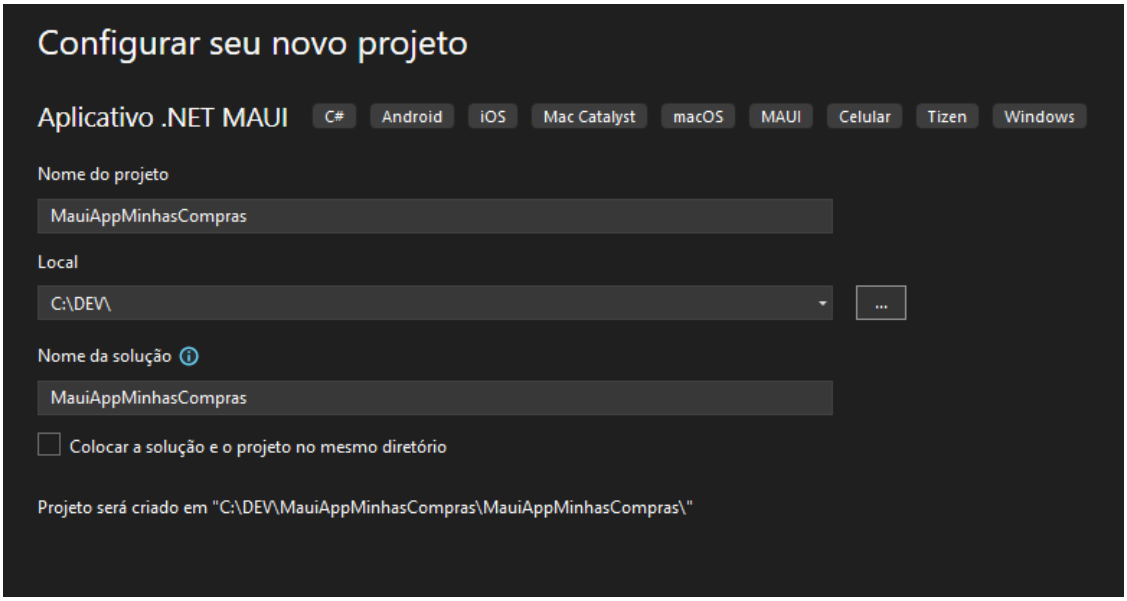
Pacotes são coleções de bibliotecas, ferramentas ou componentes que podem ser adicionados a projetos para simplificar o desenvolvimento de funcionalidades específicas. Em projetos .NET, os pacotes são gerenciados por meio do NuGet, que é um gerenciador de pacotes oficial do .NET permitindo aos desenvolvedores encontrar, instalar e atualizar bibliotecas e ferramentas em projetos. Ele automatiza a configuração de dependências e simplifica a inclusão de funcionalidades externas no código.

Ao utilizar o SQLite em um projeto .NET MAUI, é necessário instalar o pacote **sqlite-net-pcl** por meio do NuGet, o que elimina a necessidade de configurar manualmente as bibliotecas do SQLite.



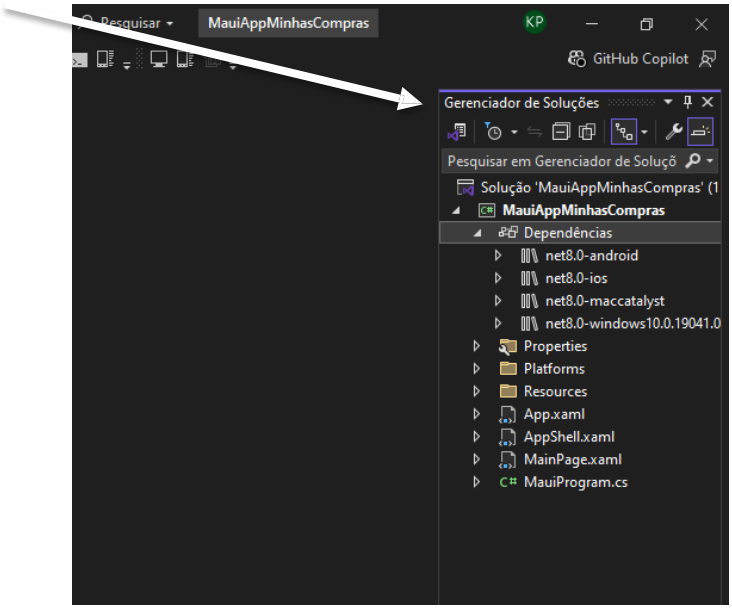
Utilize ferramentas de Inteligência Artificial para pesquisar mais sobre pacotes do .NET MAUI. Pequise sobre definição e exemplos de uilização, principalmente referente a banco de dados.

Para melhor exemplificar a utilização do pacote **sqlite-net-pcl**, vamos iniciar o projeto desta aula, denominado MauiAppMinhasCompras. Para tal, inicie o VisualStudio e crie um novo projeto .NETMAUI, conforme aprendemos no módulo anterior.

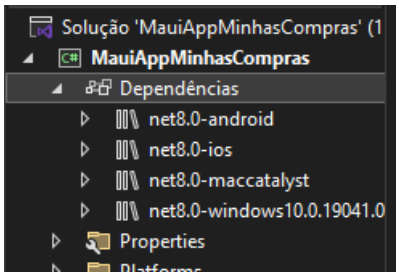


Após criação do projeto aguarde carregamento do diretório e execute o projeto uma primeira vez para certificar que está tudo correto.

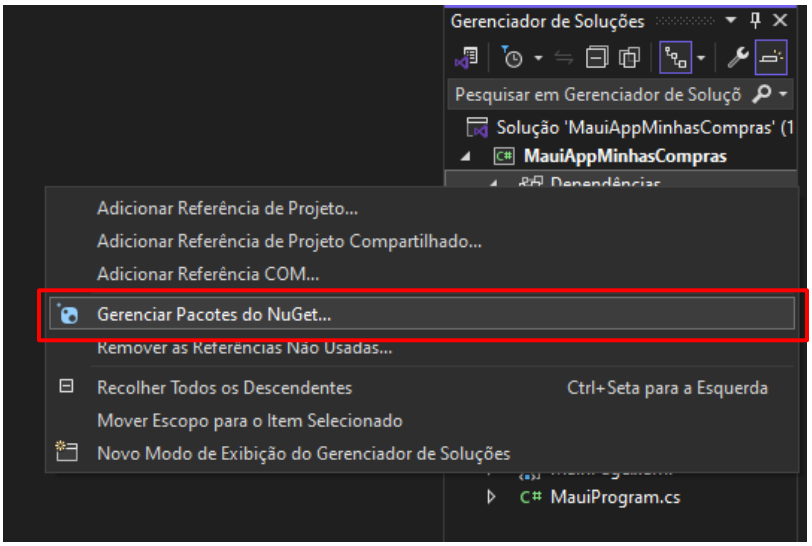
Logo após verificar os diretórios do projeto por meio do **Gerenciador de Soluções**.



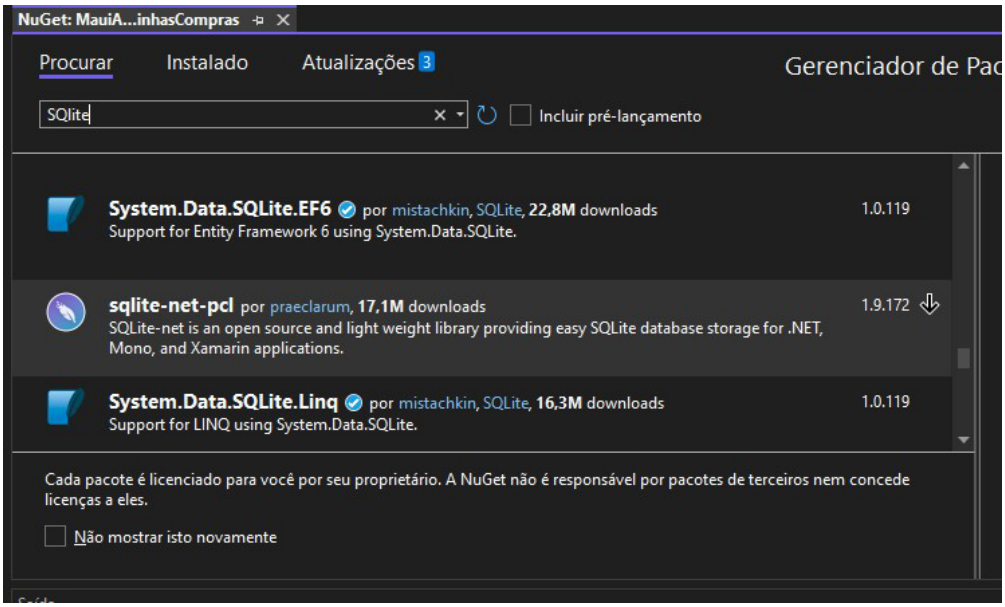
Em Gerenciador de Soluções localize o gerenciador de Dependências.



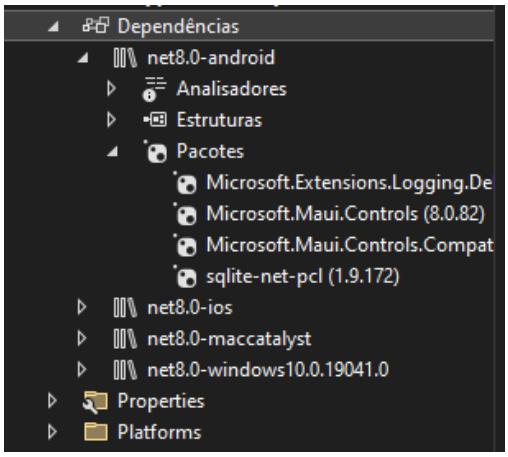
Clique com o botão direito e selecione a opção **Gerenciar Pacotes do NuGet...**



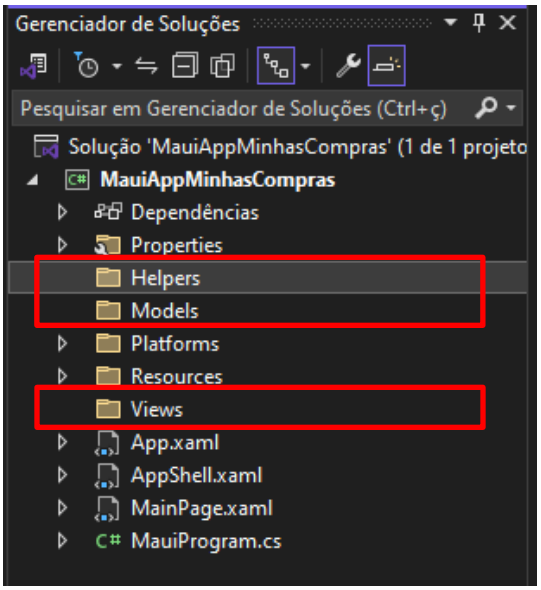
Na nova janela carregada selecione a opção **Procurar,** na caixa de busca digite SQLite, navegue entre as opções até encontrar a opção **sqlite-net-pcl** e clique em Instalar.



Após processo de instalação, certifique-se que o processo foi realizado corretamente, voltando ao menu **Dependências,** selecione a opção correspondente a Android, pacotes.

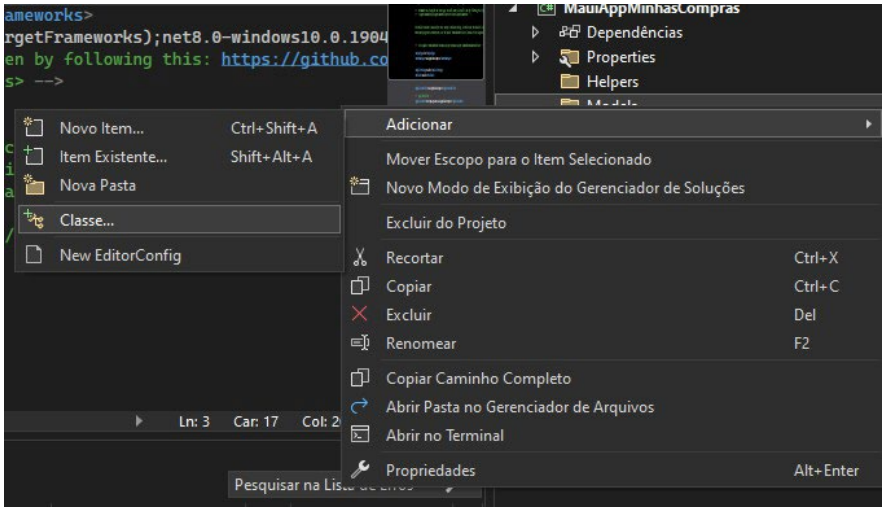


Agora que já foi instalado o pacote, vamos fazer uma organização modular das pastas do projeto, que promove a separação de responsabilidades no código. Isso facilita o desenvolvimento, a manutenção e a escalabilidade do projeto. Para tal, vamos criar 3 novas pastas no projeto, sendo: **Models**, **Views** e **Helpers**.

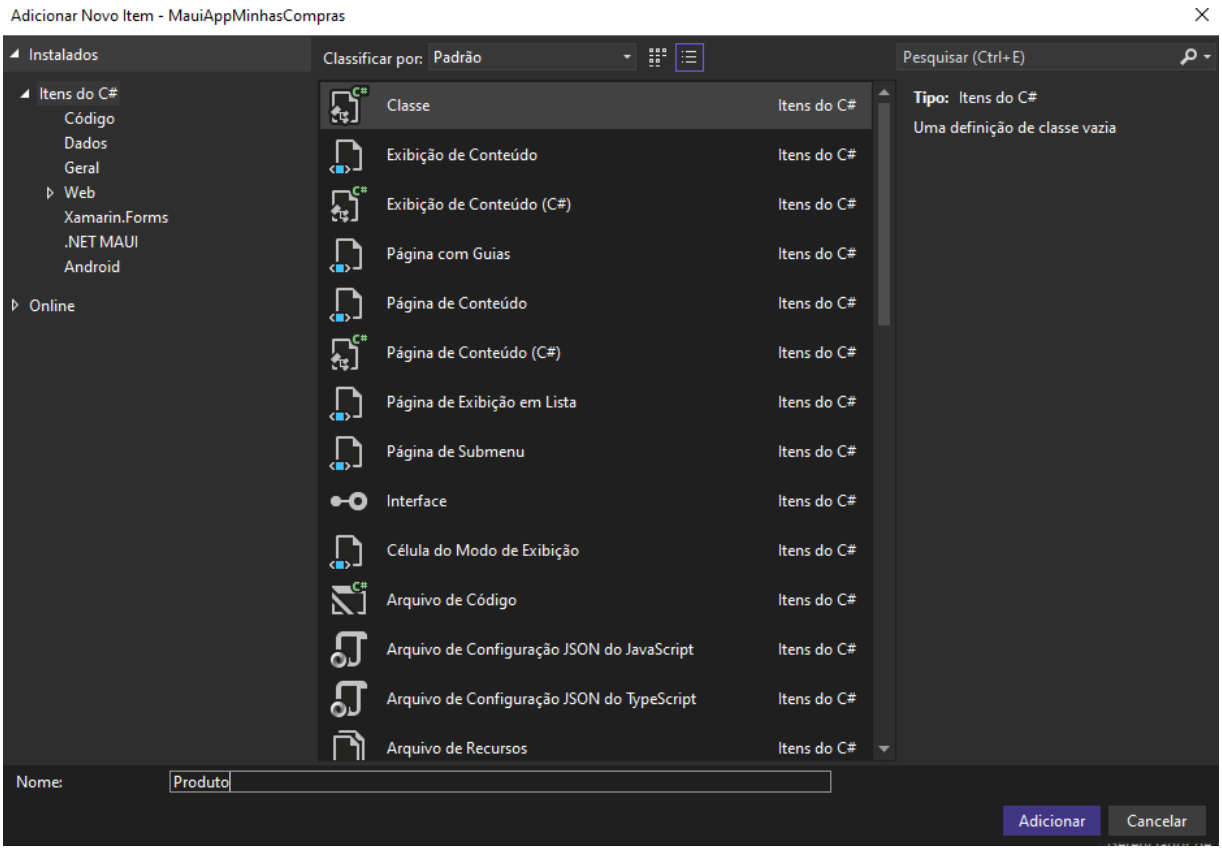


A pasta **Models** armazena as classes que representam os dados e as regras de negócio do aplicativo. Os **Models** descrevem a estrutura dos dados, incluindo propriedades e atributos, e são usados para interagir com o banco de dados ou serviços externos. Já a pasta **Views** contém as páginas ou interfaces de usuário, geralmente implementadas em XAML no .NET MAUI. Essas páginas são responsáveis por exibir os dados aos usuários e capturar as interações, como cliques ou entradas de texto. Por fim, a pasta **Helpers** armazena classes utilitárias ou auxiliares que encapsulam funcionalidades comuns ou reutilizáveis. No contexto deste projeto, a classe **SQLiteDatabaseHelper** na pasta **Helpers** gerencia a conexão com o banco de dados SQLite e fornece métodos para realizar operações CRUD.

Vamos iniciar adicionando a classe **Produto** na pasta **Models.** Clique com o botão direito sobre a pasta, no menu de opções selecione Adicionar, opção Classe.

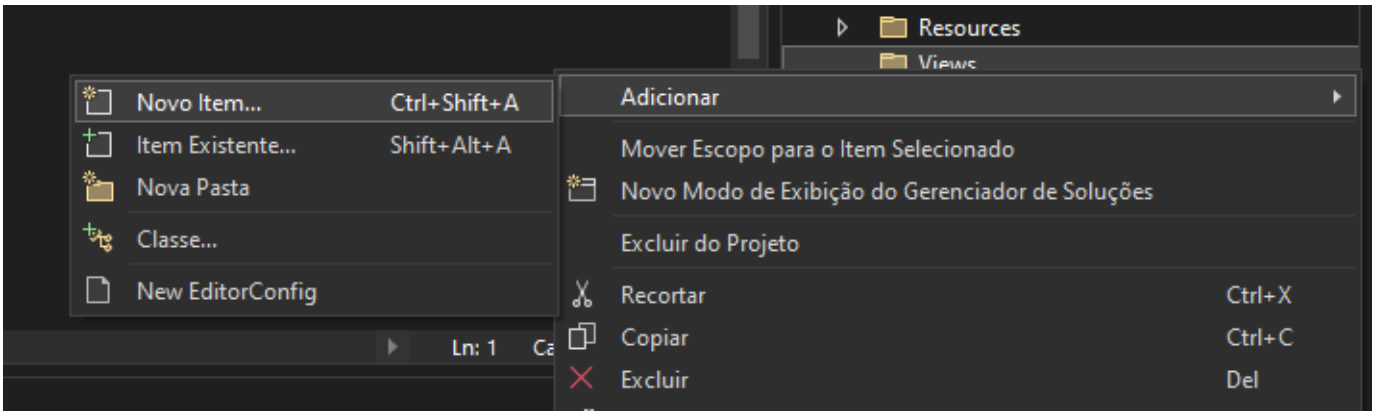


Defina o nome **Produto** e clique no botão **Adicionar.**

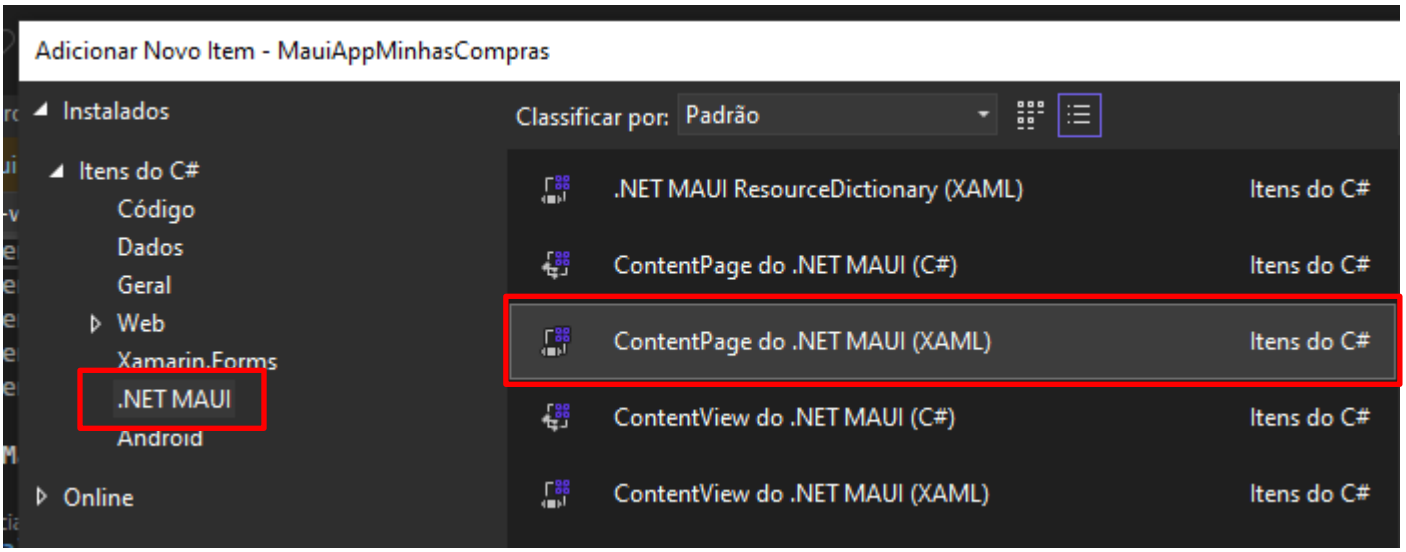


Utilizando o mesmo processo adicione na pasta **Helpers** uma nova classe denominada **SQLiteDatabaseHelper.**

Na pasta **Views** é necessário adicionar a interface do projeto, para isso é necessário selecionar a opção **ContentPage do .NETMAUI (XAML).** Clique sobre a pasta **Views**, opção Adicionar, opção **Novo Item**.



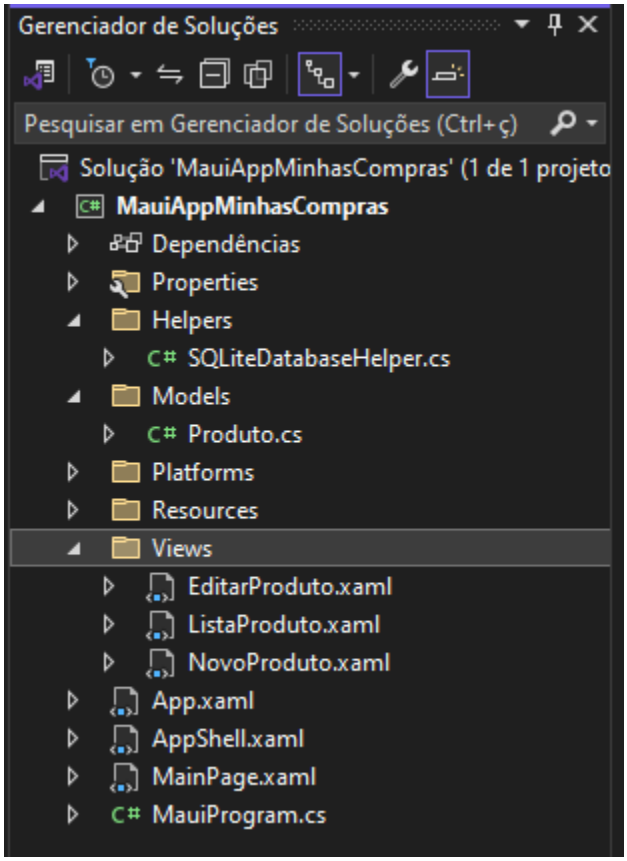
Em Instalados selecione a opção **.NETMAUI,** em seguida no menu de opções selecione **ContentPage do .NET MAUI (XAML).**



Defina o nome **EditarProduto** e clique no botão **Adicionar** para finalizar.

Repita o processo e adicione dois novos arquivos **NovoProduto** e **ListaProduto.**

O diretório de pastas ficará da seguinte forma:



Usings

Os using no C# funcionam como atalhos que nos permitem acessar funcionalidades específicas de bibliotecas e frameworks sem precisar repetir todo o caminho de seus namespaces. Imagine que você está organizando uma caixa de ferramentas: os using são como etiquetas que ajudam a localizar rapidamente a ferramenta que você precisa.
No projeto MauiAppMinhasCompras vamos utilizar o using para adicionar o SQLite a classe.

using SQLite;

Isso significa que estamos dizendo ao programa: "Quero usar as ferramentas e classes relacionadas ao SQLite que estão disponíveis no namespace SQLite". Isso inclui elementos como SQLiteConnection, que nos permite criar conexões com o banco de dados, ou atributos como [PrimaryKey] para definir a chave primária de uma tabela.
Sem os using, teríamos que escrever o caminho completo toda vez que quiséssemos usar algo dessa biblioteca, como:

SQLite.SQLiteConnection connection = new SQLite.SQLiteConnection(dbPath);

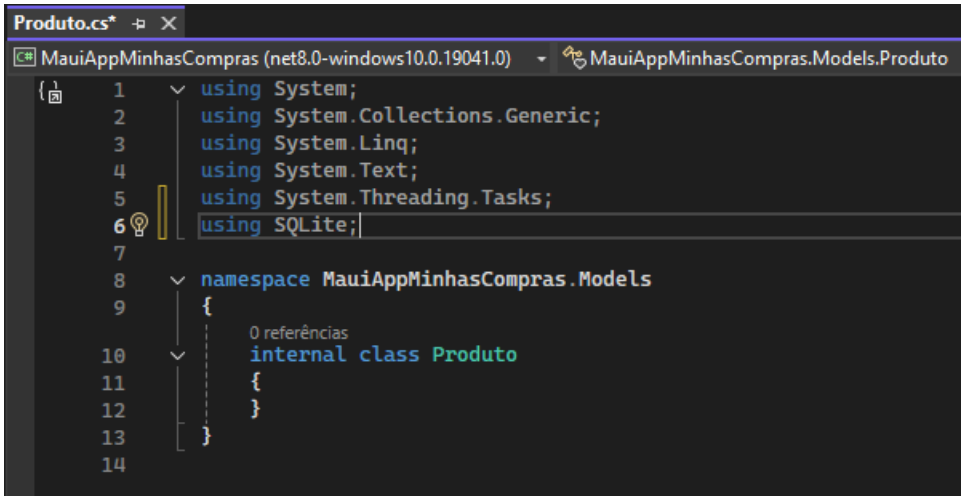
Com o using, podemos escrever:

SQLiteConnection connection = new SQLiteConnection(dbPath);

Além de deixar o código mais limpo, os using ajudam o compilador a entender quais dependências estão sendo utilizadas no projeto. Isso também melhora a legibilidade e reduz a possibilidade de erros ao digitar caminhos longos ou complexos.

Ao adicionar **using SQLite** no projeto é permitido acessar diretamente classes e métodos relacionados ao SQLite, como SQLiteConnection ou atributos como [PrimaryKey], sem precisar escrever SQLite.SQLiteConnection toda vez.

Vamos adicionar o **using SQLite** na classe Produto.



Como pode-se observar, por padrão, as classes trazem alguns using já na sua criação, porém enquanto não utilizados é importante observar que ficam na cor cinza.

Annotations do SQLite

As annotations do SQLite em projetos .NET MAUI são atributos que aplicamos as classes modelo para definir como essas classes serão mapeadas para tabelas e colunas no banco de dados SQLite. Esses atributos são fornecidos pela biblioteca **SQLite-net-pcl** e ajudam a especificar detalhes como chaves primárias, autoincremento, nomes de colunas, entre outros.

Alguns dos Principais Atributos:

`[Table("TableName")]:` utilizado para definir o nome da tabela do banco de dados, caso não seja especificado, o nome da classe será usado como nome da tabela.

```
[Table("Cadastro")]
public class Produto
{
    public int Id { get; set; }
    public string Nome { get; set; }
}
```

`[PrimaryKey]:` especifica que o campo é a chave primária da tabela.

```
public class Produto
{
    [PrimaryKey]
    public int Id { get; set; }
}
```

`[AutoIncrement]:` faz com que o valor do campo seja gerado automaticamente (incrementado) pelo banco de dados, este atributo pode ser utilizado em conjunto com o [PrimaryKey].

```
public class Produto
{
    [PrimaryKey, AutoIncrement]
    public int Id { get; set; }
}
```

`[Column("ColumnName")]:` utilizado para especificar o nome da coluna no banco de dados, sendo utilizado caso seja necessário que o nome da coluna no banco seja diferente do nome da propriedade.

```
public class Produto
{
    [PrimaryKey, AutoIncrement]
    public int Id { get; set; }

    [Column("NomeCompleto")]
    public string Nome { get; set; }
}
```

[Ignore]: utilizado para indicar que o campo ou propriedade não será incluído no banco de dados. Muito utilizado para propriedades que são apenas para lógica de aplicação e não devem ser persistidas.

```
public class Produto
{
    public int Id { get; set; }

    public string Nome { get; set; }

    [Ignore]
    public string TempData { get; set; }
}
```

[Unique]: Garante que os valores dessa coluna sejam únicos no banco de dados.

```
public class Produto
{
    [PrimaryKey, AutoIncrement]
    public int Id { get; set; }

    [Unique]
    public string Email { get; set; }
}
```

[MaxLength(length)]: define o tamanho máximo de caracteres para o campo.

```
public class Produto
{
    public int Id { get; set; }

    [MaxLength(50)]
    public string Nome { get; set; }
}
```

[NotNull]: Indica que o campo não pode ser nulo.

```
public class Produto
{
    public int Id { get; set; }

    [NotNull]
    public string Nome { get; set; }
}
```

A codificação completa da classe Produto deve ficar da seguinte forma:

```
using SQLite;

namespace MauiAppMinhasCompras.Models
{
    public class Produto
    {
        [PrimaryKey, AutoIncrement]
        public int Id { get; set; }
        public string Descricao { get; set; }
        public double Quantidade { get; set; }
        public double Preco { get; set; }
    }
}
```

Assista o vídeo apresentando o passo a passo para criação do aplicativo:

Disponível em: https://www.youtube.com/watch?v=sOnjJDD6kQo

SQLite: Guia de Persistência para .NET MAUI

O QUE É

Biblioteca embutida, autossuficiente e sem servidor que armazena dados em um único arquivo.

Zero Configuração
Não requer instalação, administração ou configuração de um processo de servidor separado.

NO .NET MAUI

sqlite-net-pcl

Integração via Pacote NuGet:
Utiliza-se o pacote sqlite-net-pcl para automatizar a configuração das bibliotecas no projeto.

ESTRUTURA DO APPMINHASCOMPRAS

AppMinhasCompras

Models
dados

Views
interface

Helpers
conexão e CRUD

SQLite

ANNOTATIONS C# (ATRIBUTOS)

```
[PrimaryKey]
[AutoIncrement]
[Table("Products")]
[Ignore]
```

Annotation	Função Visual
[PrimaryKey]	Define o campo como a chave primária da tabela.
[AutoIncrement]	Gera valores automáticos para campos numéricos.
[Ignore]	Indica que a propriedade não deve ser salva no banco.

TIPOS DE DADOS SUPORTADOS

INTEGER
Armazena números inteiros.

REAL
Armazena números de ponto flutuante.

TEXT
Armazena strings de texto.

BLOB
Armazena dados binários grandes.

NULL
Representa um valor nulo.

NotebookLM